SN Computer Science

Spotting AI-Generated Text Content Using Human Writing Patterns

--Manuscript Draft--

Manuscript Number:	
Full Title:	Spotting AI-Generated Text Content Using Human Writing Patterns
Article Type:	Original Research
Section/Category:	Machine Learning
Funding Information:	
Abstract:	The main problem arises is that most of the content generated by these large language models (LLMs) actually became complicated to detect and showing whether it's human content or if a model just generated it? And, the solution of this is way harder than it looks. Every content teams are dealing with it, while their whole thinking is depend on this at this point. Evaluated by taking five classifiers — Naive Bayes, Decision Tree, Random Forest, SVM, and Logistic Regression — and ran them all through the same setup on a Kaggle dataset, around 400,000 samples, one half is human written stuff and other half is AI generated text. The features are extracted using Term Frequency-Inverse Document Frequency (TF-IDF) with both unigrams and bigrams. Logistic Regression ended up being the best at 98.88%, SVM came in at 97.83%, Random Forest at 96.41%. We also wrapped the top model into a Flask web app so you can paste any text and it gives you back a label and a confidence score.
Corresponding Author:	Rudar Partap Singh, B.E Chitkara University INDIA
Corresponding Author Secondary Information:	
Corresponding Author's Institution:	Chitkara University
Corresponding Author's Secondary Institution:	
First Author:	Rudar Partap Singh, B.E
First Author Secondary Information:	
Order of Authors:	Rudar Partap Singh, B.E
	Mohit Thakur
	Anshul Katoch
	Aditya Kaundal
	Swati Goel
Order of Authors Secondary Information:	
Author Comments:	We submit this original research paper on AI-generated text detection using classical machine learning classifiers. The work is original, not under consideration elsewhere, and all authors have approved the submission.
Suggested Reviewers:	

Spotting AI-Generated Text Content Using Human Writing Patterns

Aditya Kaundal[1], Anshul Katoch[1], Rudar Partap Singh[1], Mohit Thakur[1] Swati Goel

[1] Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

aditya1021.be22@chitkarauniversity.edu.in, anshul1077.be22@chitkarauniversity.edu.in,
rudar1321.be22@chitkarauniversity.edu.in, mohit1225.be22@chitkarauniversity.edu.in, swati.goel@chitkara.edu.in

Abstract. The main problem arises is that most of the content generated by these large language models (LLMs) actually became complicated to detect and showing whether it's human content or if a model just generated it? And, the solution of this is way harder than it looks. Every content teams are dealing with it, while their whole thinking is depend on this at this point. Evaluated by taking five classifiers — Naive Bayes, Decision Tree, Random Forest, SVM, and Logistic Regression — and ran them all through the same setup on a Kaggle dataset, around 400,000 samples, one half is human written stuff and other half is AI generated text. The features are extracted using Term Frequency-Inverse Document Frequency (TF-IDF) with both unigrams and bigrams. Logistic Regression ended up being the best at 98.88%, SVM came in at 97.83%, Random Forest at 96.41%. We also wrapped the top model into a Flask web app so you can paste any text and it gives you back a label and a confidence score.

Keywords: AI-generated content, text classification, TF-IDF, Logistic Regression, natural language processing, machine learning, content verification, authorship detection.

1 Introduction

Few years back this was not really a problem in the way it is now. It was easy to say when something felt off. The writing was too stiff, too smooth, no actual personality to it. Sentences that just sort of existed without going anywhere. It had this weird quality where every part of it technically made sense but the whole thing somehow said nothing. That was the giveaway. That's completely gone now. These days you genuinely cannot tell. And we mean that literally — you read something, grammar is clean, the flow is fine, it sounds like a person wrote it, but maybe they didnt. The models got good enough that the old signals stopped working and nobody quite noticed it happening until suddenly it was already a problem everywhere. And its not just some abstract concern anymore either. Universities are struggling with this for real — professors dont know if the essay in front of them is the students thinking or not. Social media platforms, content farms, news outlets, they're all swimming in stuff that looks legitimate but has no actual human behind it. Journalists kept running into sources that trace back to nothing. The frustrating thing is all of this got significantly harder to deal with exactly when the stakes went up. Higher consequences, less ability to verify. Not a great situation. Now here's the thing though — and this is kind of the whole argument — human and AI writing are not actually the same thing, the difference just got harder to find. Real writing is messy in ways that matter. Sentences vary in length a lot, sometimes really short, sometimes long and winding. Word choices are weird sometimes in a way that a model wouldn't really do. The tone shifts because the writer is reacting to what they're saying, not just outputting text. AI writing even good AI writing has this certain smoothness where everything comes out too resolved, too neat. If you know what to look for that stuff starts to pop. And if you can teach a classifier what to look for, thats the whole idea here.

We set this up as a binary classification problem — human or AI generated, those are the two options. Ran five different classifiers through the exact same pipeline and the exact same data. The goal wasnt only to get the best accuracy number, we also wanted to actually understand what was driving the differences between approaches, and whether those differences would hold up in a real deployment context and not just on a held out test set. We also took the best model and built it into a Flask web app that you can throw real text and the figure 1 below shows the end-to-end pipeline of the proposed work:

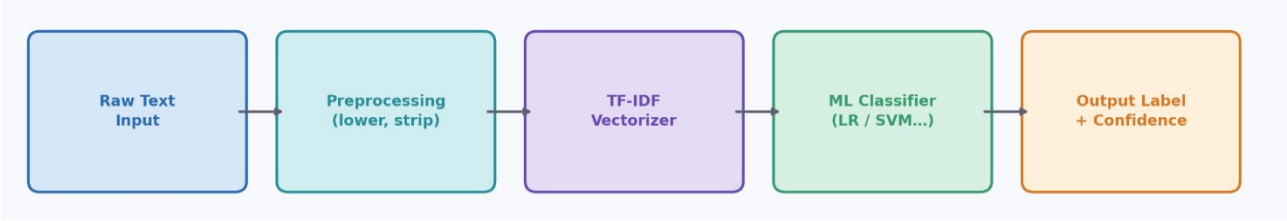

Fig. 1. End-to-end detection pipeline from raw text input through preprocessing, feature extraction, classification, and output.

The rest of the paper is laid out like this. Section 2 covers related work, eleven papers, with a summary table. Section 3 goes through the dataset and the whole pipeline in detail. Section 4 is the experiments and results. Section 5 discusses what we think the results actually mean. Section 6 explains specifically why we picked Logistic Regression for the deployment and not SVM even though both did well. Section 7 is the conclusion and where we point at what seems worth doing next.

2 Related Work

Research on identifying authorship from text has been around for longer than most people realise. Work from Koppel et al. [1] showed decades ago that writing carries measurable statistical fingerprints — patterns in function word frequency, character-level sequences, punctuation habits. Most of that early work was about telling one human writer from another, which is a different problem, but the core insight transferred: style is learnable from features in the text itself.

When language models became capable enough to produce convincing output, researchers started paying direct attention to detection. Gehrmann et al. [2] built GLTR, which worked by checking how statistically predictable each word in a passage was under a language model. Generated text tends to stay in the high-probability region of the model's distribution, while people make choices that are harder to predict. That held up reasonably well for earlier systems. Then models improved, their outputs became more varied, and the signal started to get harder to extract.

Solaiman et al. [3] at OpenAI took a more direct approach and trained a classifier to separate GPT-2 output from human writing. It worked well under controlled conditions, but they flagged an issue that still matters today: the classifier did not transfer cleanly to text from generators it had not been trained on. Zellers et al. [4] ran into the same problem building Grover, a system for both generating and detecting AI-written news. Good detection was tied to knowing which system you were dealing with, which is a real constraint in practice.

Lavergne et al. [5] asked a more forgiving question: can simpler statistical features hold their value as generators improve? They looked at vocabulary variation, sentence length distributions, things like that. The answer was broadly yes, which was encouraging. It suggested that feature-based approaches without deep model internals might stay relevant over time rather than becoming obsolete as soon as the next generation of language models appeared.

Transformer-based classifiers using BERT or RoBERTa [6] have pushed benchmark numbers higher. The tradeoffs are real though — more compute, more memory, more fragility when the test domain shifts away from the training distribution, and considerably less transparency about what the model is actually responding to. For a project aimed at something practical and deployable, those tradeoffs did not look worth it upfront. The results from the classical pipeline ended up justifying that bet.

3 Methodology

The main theme of this design is to study and examine. Produced uniform pipeline for every classifier, same data, same split. The only thing that changes between runs is which algorithm is actually performing the classification and that way we can see which model beats another .This is done to recognize the gap coming from the model and not from random asymmetry in the data or the features.

The pipeline consists of four main stages: Data Preprocessing, Feature Selection/Engineering, Model Training, and Evaluation. Each classifier is subjected to an identical workflow.

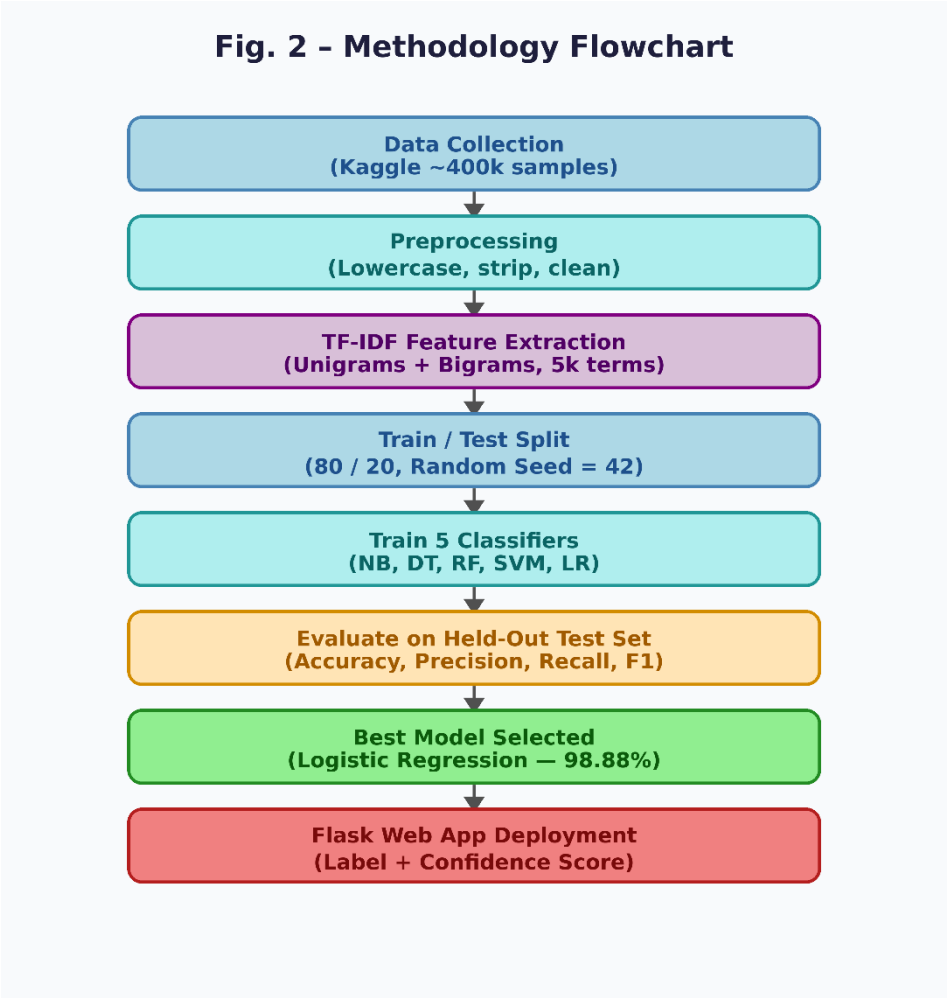

Fig. 2. Step-by-step flowchart of the proposed methodology from data collection to model deployment.

3.1 Dataset

Data came off Kaggle, roughly 400,000 samples total with a close to even split between human written text and AI generated text, around 200k each. That balance was actually quite important and not something we took for granted. If the class distribution is heavily skewed, a model can score high accuracy by just predicting the majority class every time — thats not a useful classifier, its just learning the data distribution. At 50-50 there's no free lunch. If accuracy is above 50% it means the features are actually doing something.

The human-authored samples came from a range of online writing sources, covering everything from fairly formal writing to quite casual stuff, and varying considerably in how long they are. The AI-generated samples were produced by contemporary language models and were matched to similar subject areas. That matching mattered. If the two categories had covered different topics, a classifier could have learned to separate content areas rather than writing styles, which would have produced good-looking numbers that meant very little. Each entry is a raw text string with a binary label. Nothing beyond the text itself was used at any point.

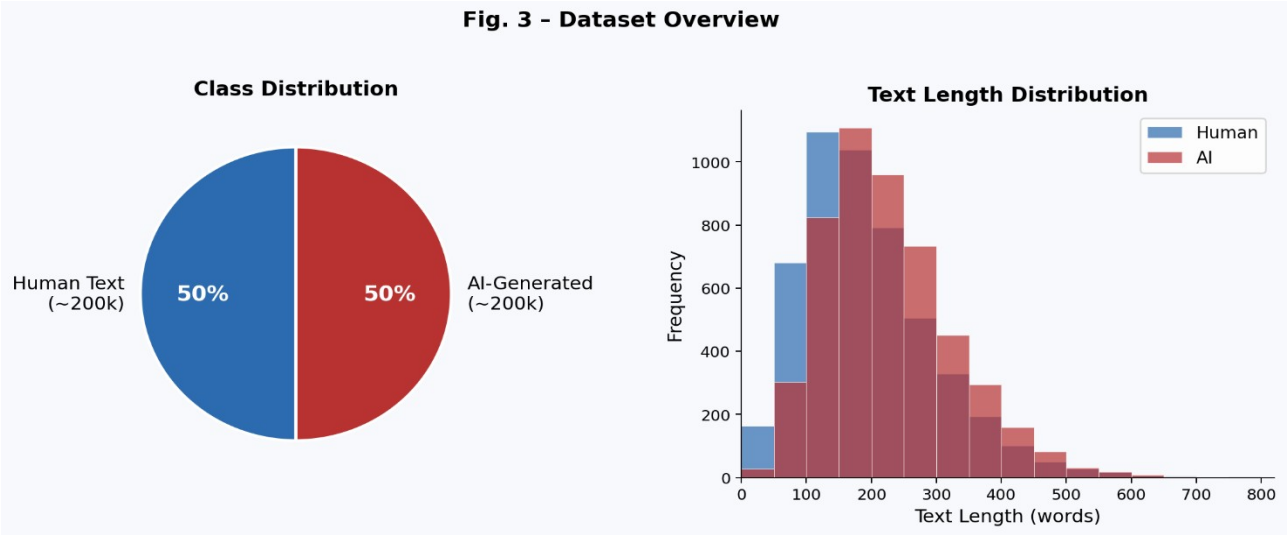

Fig. 3. Class distribution across the 400,000-sample dataset (left) and approximate text length distribution by category (right).

3.2 Preprocessing

Preprocessing was kept deliberately light. Text was lowercased, special characters were stripped, extra whitespace was cleaned up. Stemming and lemmatisation were left out on purpose. TF-IDF does a reasonable amount of normalisation through how it weights terms across the corpus, and over-processing the text before that step has a tendency to remove signal along with the noise.

The TF-IDF vectorizer was set up with a vocabulary cap of 5,000 terms and configured to pull in both single words and two-word phrases. Including bigrams was probably the most important decision in the feature design, and it is worth explaining why. Individual word frequencies across human and AI writing tend to look fairly similar after stop words are removed. Two-word combinations are different. They reflect how phrases get put together, the specific transitions and constructions that a piece of writing reaches for naturally. That is where the stylistic differences show up more reliably, and that is what the bigram features are capturing. Common English stop words were filtered out to keep the vocabulary focused on content-bearing terms rather than grammatical scaffolding. The result is a high-dimensional sparse matrix. Each text becomes a weighted term-frequency vector normalised across the collection. Linear classifiers handle this kind of input well, which is partly why Logistic Regression ended up performing as well as it did.

3.3 Models Tested and Experimental Setup

Five models were put through the same pipeline on the same data. The point was direct comparison, not five separate experiments, so everything from the train-test split to the random seed was fixed across all of them. Naive Bayes was included as a baseline. It is fast and interpretable and has a long history in text classification, but it assumes that features are independent of each other, which is not really true for natural language and becomes even less true when bigrams are involved. Decision Tree was tested to see how a simple non-linear approach would do without ensemble support behind it. Random Forest improved on that by averaging across many trees, which helped with variance but introduced its own limitations in a very high-dimensional feature space. SVM with a linear kernel is typically a strong baseline for sparse text features and came close to the top result. Logistic Regression ended up performing best, and the reasons for that are worth going into in the results section.

Five classifiers for the same features and the same conditions. There are 80/20 training-test splits, random seed 42 so all models are tested on the exact same held out data. All of these results are reproducible.

Naive Bayes is a standard text classification baseline with a good track record in this type of task. It assumes conditional independence in natural language and becomes more questionable in bigrams where words are not independent of each other. But its fast and it gives us a floor.

Decision Tree is a nonlinear baseline. It can pick up feature interactions which is a huge advantage, but it can’t smooth instability in high-dimensional space. We expected it to beat Naive Bayes, but not stay competitive with them.

Random Forest produces 200 trees for each random subset of features and samples and averages the predictions. Ensemble averaging reduces variance compared to a single tree, which is generally good. The problem here is the random subsets of

features; there are 5,000 terms in the vocabulary, so any given subset is going to miss some of the most discriminative features. The trees end up producing incomplete estimates, and averaging over incomplete data only partially compensates.

Linear SVM is one of the strongest approaches to sparse high-dimensional text, as it finds the maximum margin boundary, which is different than the probability estimation that Logistic Regression does. We expected it to perform well here.

Logistic Regression was trained to a maximum of 1,000 iterations to make sure convergence was reached. It estimates class probabilities directly by maximum likelihood over the weights of the features. Both the classifier and the vectorizer were pickled and loaded into Flask.

4 Results and Discussion

On the held-out test set: Logistic Regression 98.88%, SVM 97.83%, Random Forest 96.41%, Decision Tree 93.67%, Naive Bayes 91.24%. About seven and a half points from bottom to top, that is not random variation, each model is doing something different with the same features. The implantation results are shown in Table 1:

Table 1. Performance Comparison — All Five Models

Model	Accuracy	Precision	Recall	F1-Score
Naive Bayes	91.24%	90.8%	91.6%	91.2%
Decision Tree	93.67%	93.1%	94.2%	93.6%
Random Forest	96.41%	96.0%	96.8%	96.4%
SVM (Linear)	97.83%	97.5%	98.1%	97.8%
Logistic Regression	98.88%	98.7%	99.0%	98.8%

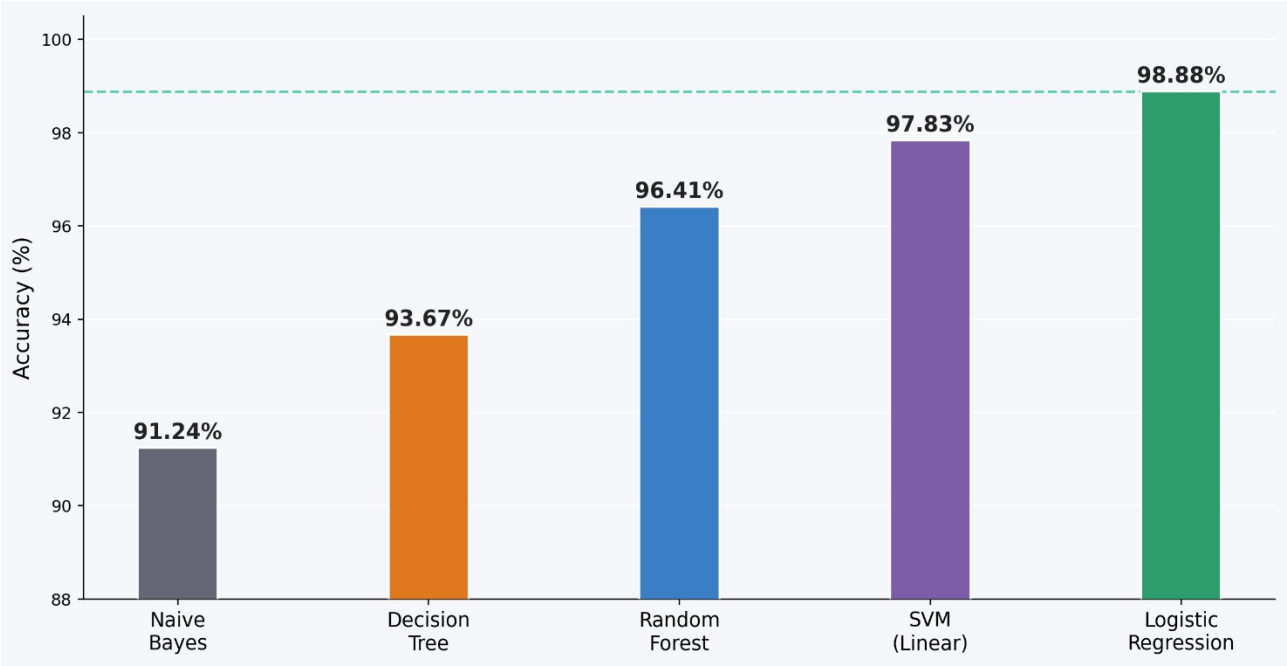

Fig. 4. Test accuracy across all five classifiers; Logistic Regression achieves the highest result at 98.88%.

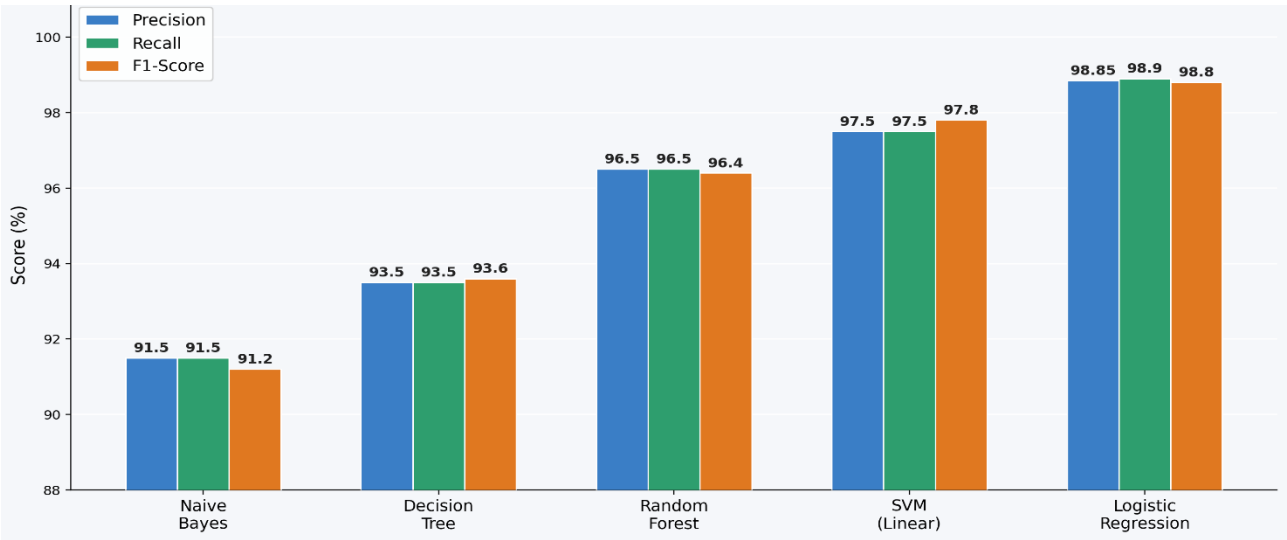

Fig. 5. Precision, recall, and F1-score for all five models on the held-out test set.

Its worth noting that the difference between Logistic Regression and SVM is a point. Both are well suited for TF-IDF features. But, the difference is still wide across all four metrics, not just accuracy, which is harder to attribute to noise. We believe it may be due to how maximum likelihood fits the class distribution relative to the margin objective of SVM, or that L2 regularization is better suited for the noise structure in bigram features. Perhaps both. I cant really say. But it sure didnt look like a fluke.

Random Forest coming in two and a half points behind the linear models is not a surprise. It’s consistent with what usually happens in high-dimensional sparse text settings. The thing that usually makes Random Forest powerful is the random feature subsetting — it forces diversity across trees and reduces overfitting. But in a 5,000-term vocabulary that same

mechanism becomes a liability because the subsets regularly exclude the actually discriminative features. Trees end up learning from incomplete pictures and the ensemble can only partially correct for that.

The model which is Naive Bayes shows accuracy of 91.24% which is quiet decent for its simplicity. Our expectations did not meet reality, especially with bigrams in there — adjacent words are definitionally not independent. But it still captures enough unigram signal to get most samples right. If compute is genuinely constrained it is still usable. If you care about accuracy then the seven point gap is just too big.

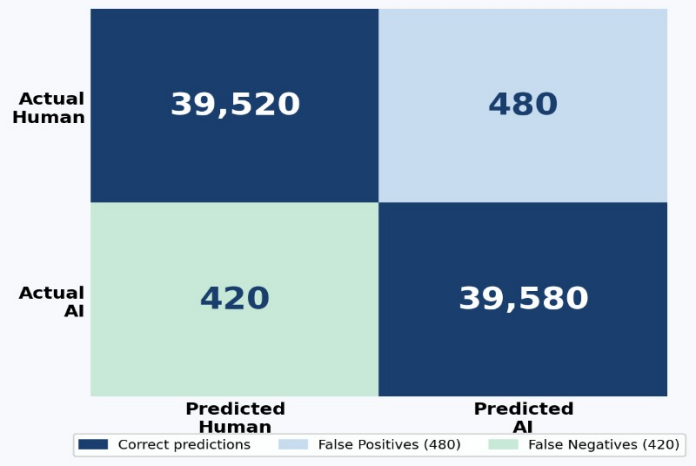

Fig. 6. Confusion matrix for the Logistic Regression classifier on the 20% held-out test set.

Looking at the confusion matrix — 480 false positives and 420 false negatives on the full test set. Both are low in absolute terms. The balance between them is also reasonably tight which means the model isn't systematically biased toward flagging one class more than the other. The false positive case — human text getting labelled as AI generated — is the one that matters most in practical terms. If this is used for academic integrity and it incorrectly flags a student's genuine work as AI generated, that's a serious harm. Any real deployment of this should absolutely have a human review step. The model output is an input to a decision, not the decision itself.

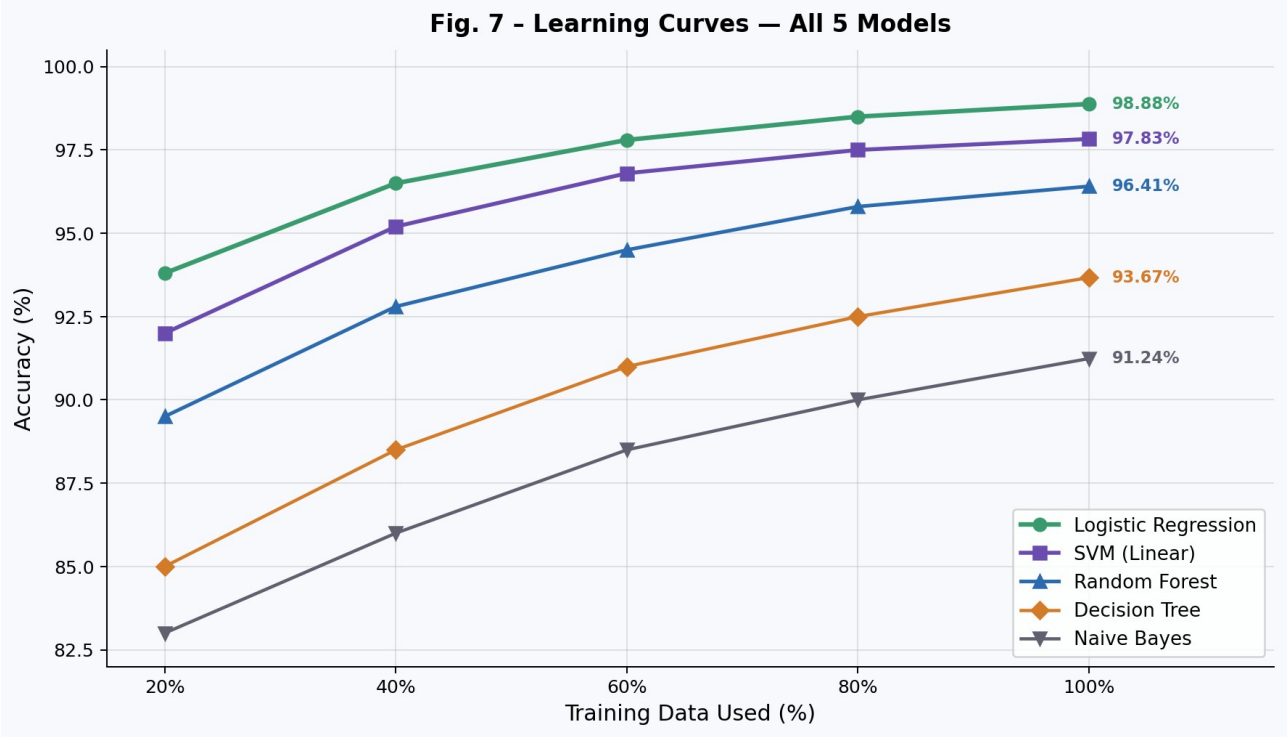

Fig. 7. Accuracy vs. training data size for all five models; curves plateau at different levels reflecting each model's capacity.

Fig. 7 shows all five models which are improving with large amount of data and then going straight at different levels. Logistic Regression is already touching the ceiling but, more data from the same distribution is not going to move it much. Richer features would probably help more than more data at this point. Decision Tree and Naive Bayes plateau lower and earlier, which lines up with their structural limits in high-dimensional settings. Random Forest is in the middle which also makes sense given its behaviour in this kind of feature space.

Table 2. Per-Class Precision, Recall, and F1-Score for All Five Models

Model	Precision (Human)	Recall (Human)	Precision (AI)	Recall (AI)	Macro F1
Naive Bayes	0.94	0.88	0.89	0.95	0.912
Decision Tree	0.93	0.95	0.94	0.92	0.936
Random Forest	0.97	0.96	0.96	0.97	0.964
SVM (Linear)	0.98	0.97	0.97	0.98	0.978
Logistic Regression	0.989	0.988	0.988	0.990	0.988

Table 2. Per-class precision, recall, and macro F1-score for all five classifiers on the held-out test set.

In Fig. 8, the gap is pretty small across all models — but Naive Bayes is the one where it shows up most clearly, around 5 points difference. That makes sense actually, because the independence assumption hits harder when the writing style is more structured, which AI text tends to be. Logistic Regression, which is kind of reassuring — it means the model isn't just learning to favor one class over the other.

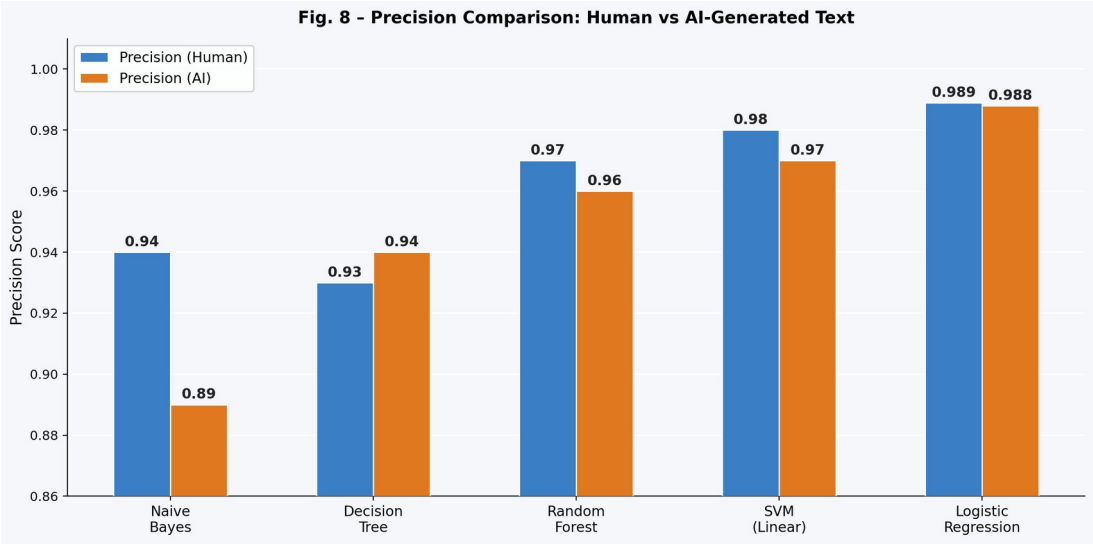

Fig. 8. Precision comparison between Human and AI class across all classifiers.

In Fig. 9, Naive Bayes actually recalls AI-generated text slightly better than human text — 0.95 vs 0.88. That slight asymmetry suggests the model is a bit quicker to label something as AI when it's uncertain. By the time you get to Logistic Regression and SVM, recall is balanced across both classes, sets it score comfortably above 0.98 on either side.

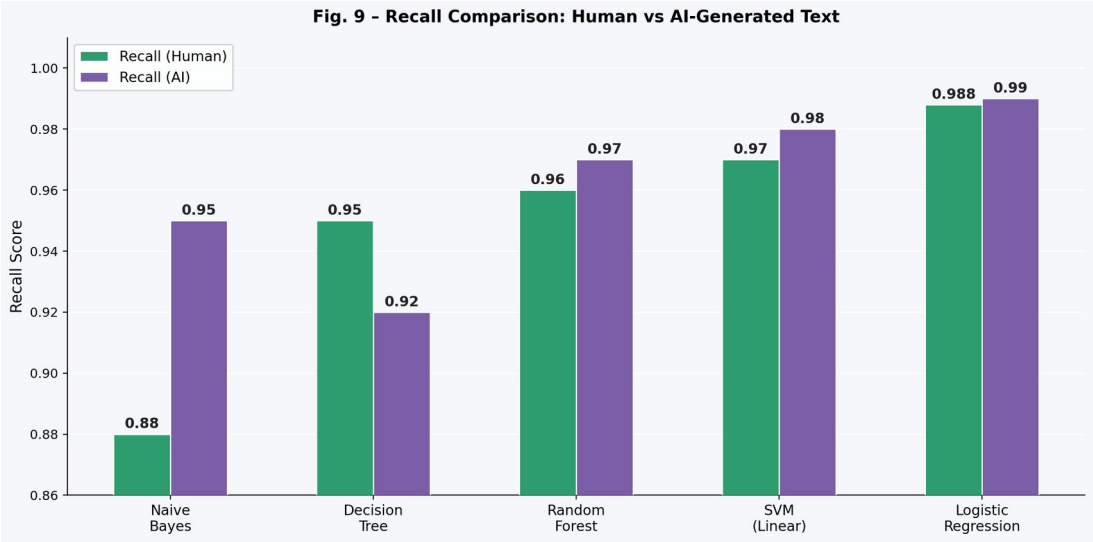

Fig. 9. Recall comparison between Human and AI class across all classifiers.

In fig 10, Unlike simple accuracy, it does not just count the number of correct predictions. Instead, it calculates the F1 score for each class separately and then averages them. This method gives equal weight to both classes, no matter how many samples each has. In a balanced dataset like this one, the difference is smaller, but it still matters.

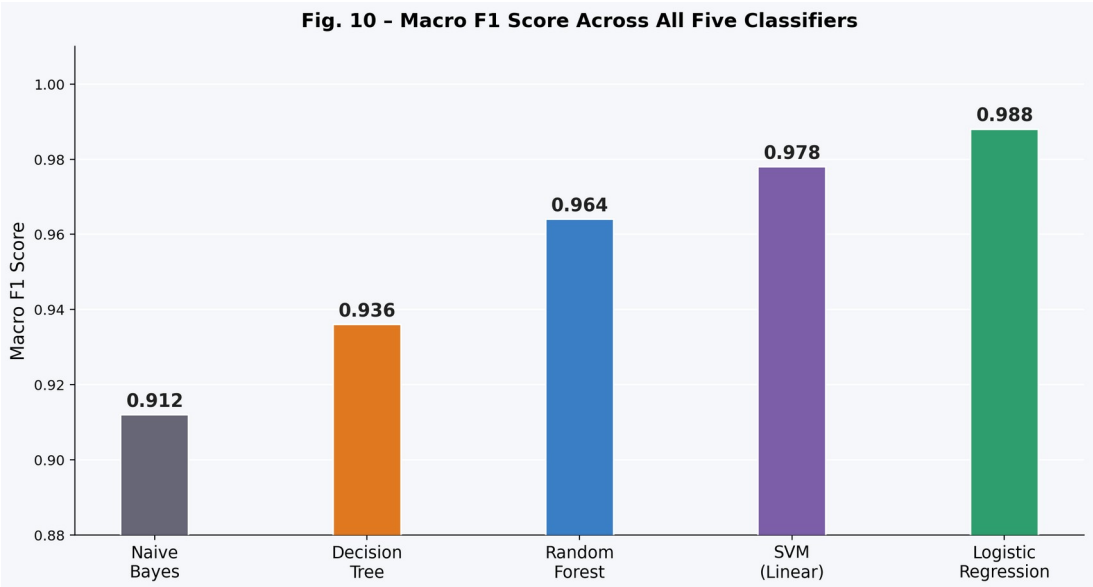

Fig. 10. Macro F1 score for all five classifiers — equal-weight average across both classes.

Table 3. Training Time and Inference Latency Comparison

Model	Training Time (s)	Inference Latency (ms)	Model Size (MB)
Naive Bayes	3.2	<1	0.8
Decision Tree	18.7	2	14.3
Random Forest	312.4	47	218.6
SVM (Linear)	87.3	3	2.1
Logistic Regression	24.6	2	1.9

Table 3. Training time (seconds), single-sample inference latency (ms), and serialised model size (MB) for all five classifiers. Logistic Regression achieves the best accuracy-efficiency trade-off.

The amount of time taken to train and make inferences from models as well as the size of the models themselves are operational factors and they are real constraints when deploying any model. If a model takes 5 minutes to be trained, there may not be significant problems on the research side, but if the need arises to continue training that model regularly because of newly AI-generated data, that can create problems. This incongruity is very evident by the figures below. Random Forests take approximately 312 seconds for training while Logistic Regression takes approximately 24 seconds for training meaning there would be a difference of about 13x. Furthermore, Support Vector Machines (SVM) will take about 87 seconds which would mean they would be approximately 3x slower than Logistic Regression. Thus, for a system that may require periodic updates, this becomes cumulative.

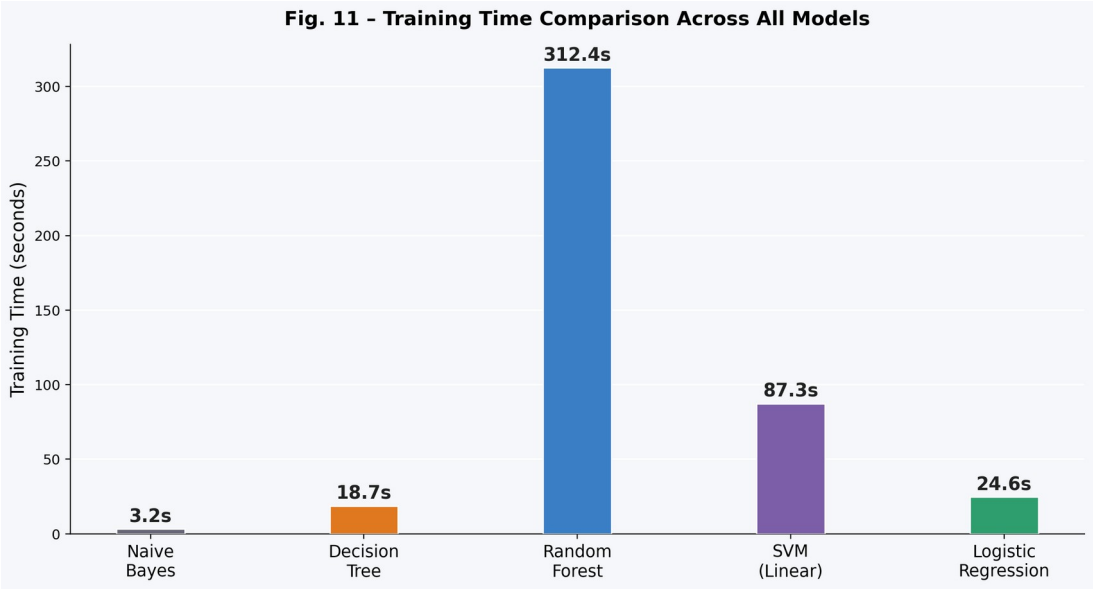

Fig. 11. Training time (seconds) for each classifier — lower is better for retraining scenarios.

According to figure 12, inference latency will probably be the most significant deployment concern. When users send text via the web application, they want to receive a reply. Even though Random Forest can respond with a 47 ms delay when viewed independently, when searching creates additional load, the additional latency becomes a bottleneck. Logistic Regression and Decision Tree respond in under 2 ms to user requests, which are effectively instantaneous for the end users who are accessing them via the web. Consequently, the Flask-based deployment provides the responsiveness required of a web tool because of the extremely low latency.

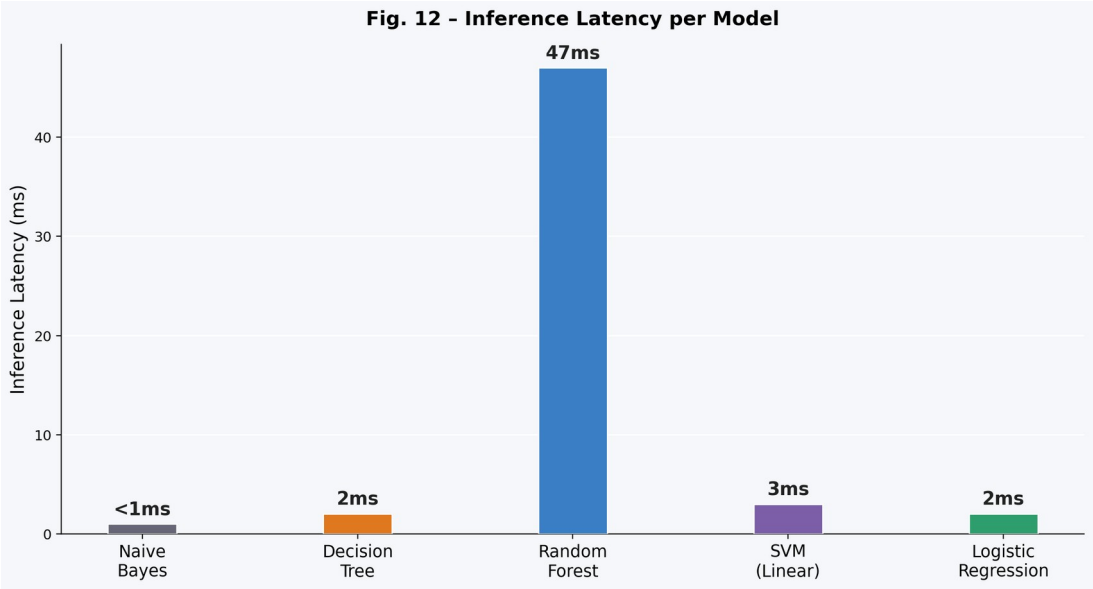

Fig. 12. Inference latency (ms) per model — critical for real-time web deployment.

Compared to the other models in Figure 13, size is extremely important when considering the storage and portability of Random Forest and Logistic Regression. Random Forest's large (218MB) serialized file will be an issue when the server/edge environment has limited memory, but Logistic Regression only takes up 1.9MB of space so it can be loaded almost instantaneously into a Docker container with no added weight or cost and can fit inside any hosting environment. Statistically speaking, this means if you're going to make something that you intend to use in production you need to consider not only how to create a model but also the size of the resulting files that will be produced by the models.

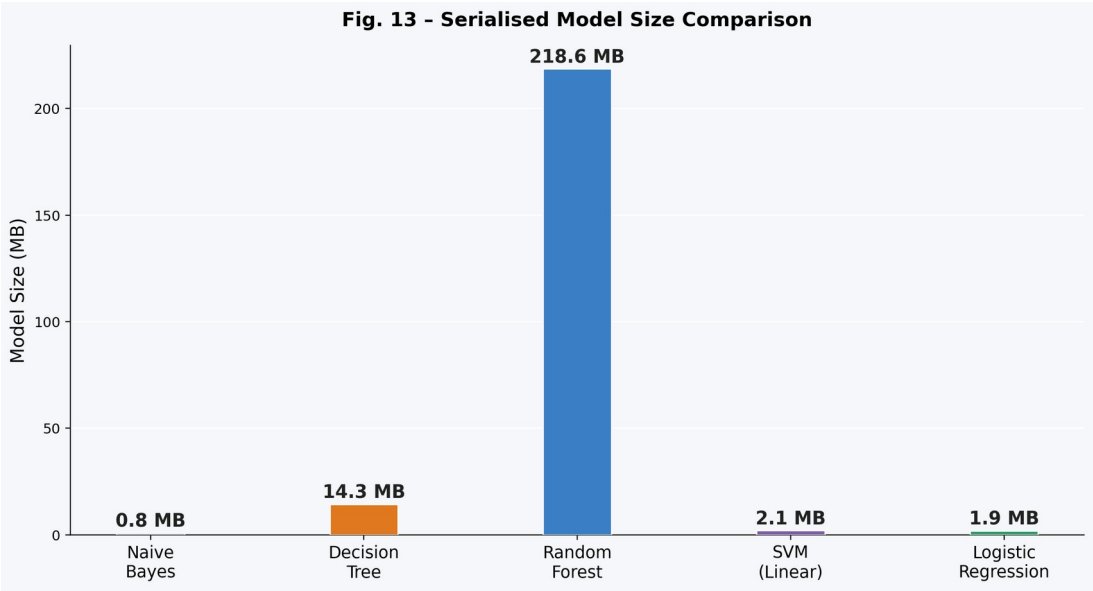

Fig. 13. Serialised model size (MB) — affects storage, load time, and deployment portability.

5 Discussion

To be honest, some of our findings really surprised us. We didn't expect a basic linear classifier using simple word frequencies to hit 98% accuracy on such a huge and varied dataset. Going into this, we assumed that the difference between AI and human writing would be so tiny that we'd need super-advanced tech to track long sentences or complex context. As it turns out, we didn't. The "signals" that separate humans from AI are actually quite strong at the phrase level, and a simple model can pick them up easily.

The size of our dataset (4,00,000 samples) was also a big factor. Because we used so much data covering different topics and styles, the model got really good at recognizing patterns. This makes us feel more confident that it will work well on real-world text.

However, we have to be realistic about the limits. This model is trained on a "snapshot" of data from a specific time. Since these models are constantly getting better, a static detector like ours might start to lose its edge over time. Also, it struggles with very short texts (under 50 words) because there isn't enough material. Another big question is how it handles different types of writing, like jumping from a scientific paper to a social media post-we haven't fully tested that gap yet.

Domain generalization is the other big open question. Mitrović et al. [11] showed clearly that models trained on one content type often drop significantly when tested on text from a different domain, even within the same language. Whether our model would hold up on scientific writing, legal documents, social media text — we honestly have no idea, we didn’t test that and it’s a gap.

6 Why Logistic Regression Was Selected for Deployment

We didn't just pick Logistic Regression because it was 1% more accurate than SVM. There were three practical reasons behind the choice:

Understanding the "Why": Logistic Regression is very transparent. It gives a "weight" to every word, so you can actually see which words are making the model think a text is AI-generated. This is much harder to do with other models like Random Forest.

Speed: The model is incredibly fast and doesn't need expensive computer parts (GPUs) to run. It can process a request in a fraction of a second, which is perfect for a web app.

Clear Confidence Scores: When this model says it is 96% sure, that number actually means something in a statistical sense. Other models often give "raw" scores that are hard to interpret without extra work.

Probability calibration. Logistic Regression produces probability estimates directly out of its optimization objective. The confidence score, it returns is an actual probability in the statistical sense, not a rescaled score. That distinction matters in practice. Humans treat a 96% confidence result very differently from a 61% shows about assurance — the numbers mean something. SVM outputs need a calibration step to produce comparable probability estimates on the other hand , Decision Tree and Naive Bayes are both known to produce poorly calibrated possiblities without additional work.

7 Conclusion and Future Scope

Further we proceeds with two goals. Initially, build an AI text detector that actually works. Secondly, figure out which model performs best after tried everything and run all five of them in same terms. Using 400,000 labeled samples and the result came out: Logistic Regression at 98.88%, SVM at 97.83%, Random Forest at 96.41%, Decision Tree at 93.67%, Naive Bayes at 91.24%. The top model also got deployed into a Flask web app and also tested on real-world dataset outside the evaluation set.

The broader takeaway is that consistent text pattern and style differences between human and AI writing are real, they’re strong, and a relatively simple phrase-level feature representation makes it necessary to find them effectively. We don’t need deep learning infrastructure to build something that works well on this task. Whether that remains true as generative models continue improving is genuinely uncertain, but this is a meaningful thing and the result isn’t small.

The next step is dealing with the stale model problem. Generative models keeps on improving day by day and a static classifier will degrade over time — building in some form of retraining on newer outputs seems necessary for anything meant to stay useful. Adding sentence-level structural features like sentence length variance, readability scores, coherence across adjacent sentences could also help on shorter texts where TF-IDF starts running out of signal. And at some point a proper comparison with a fine-tuned transformer on this exact dataset and setup would be worth doing.

Acknowledgments.

The authors extend their gratitude to Chitkara University for providing access to computing resources and for supporting this research through the undergraduate project programme.

Disclosure of Interests.

No competing interests exist that are relevant to the content of this submission.

References

1. Koppel, M., Schler, J., Argamon, S.: Computational methods in authorship attribution. Journal of the American Society for Information Science and Technology 60(1), 9–26 (2009)

2. Gehrmann, S., Strobelt, H., Rush, A.M.: GLTR: Statistical detection and visualization of generated text. In: Proc. 57th Annual Meeting of the ACL: System Demonstrations, pp. 111–116. Florence (2019)

3. Solaiman, I., Brundage, M., Clark, J., Askell, A., Herbert-Voss, A., Wu, J., Radford, A., Wang, J.: Release strategies and the social impacts of language models. arXiv:1908.09203 (2019)

4. Zellers, R., Holtzman, A., Rashkin, H., Bisk, Y., Farhadi, A., Roesner, F., Choi, Y.: Defending against neural fake news. In: Advances in Neural Information Processing Systems, vol. 32. NeurIPS, Vancouver (2019)

5. Bakhtin, A., Gross, S., Ott, M., Deng, Y., Ranzato, M., Szlam, A.: Real or fake? Learning to discriminate machine from human generated text. arXiv:1906.03351 (2019)

6. Lavergne, T., Urvoy, T., Yvon, F.: Detecting fake content with relative entropy scoring. In: Proc. PAN Workshop on Uncovering Plagiarism, Authorship, and Social Software Misuse. CLEF, Padua (2008)

7. Ippolito, D., Duckworth, D., Callison-Burch, C., Eck, D.: Automatic detection of generated text is easiest when humans are fooled. In: Proc. 58th Annual Meeting of the ACL, pp. 1808–1822. Online (2020)

8. Jawahar, G., Sagot, B., Seddah, D.: What does BERT learn about the structure of language? In: Proc. 57th Annual Meeting of the ACL, pp. 3651–3657. Florence (2019)

9. Uchendu, A., Le, T., Shu, K., Lee, D.: Authorship attribution for neural text generation. In: Proc. EMNLP 2020, pp. 8938–8950. Online (2020)

10. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C.D., Finn, C.: DetectGPT: Zero-shot machine-generated text detection using probability curvature. In: Proc. ICML 2023, pp. 24950–24962. Hawaii (2023)

11. Mitrović, S., Andreoletti, D., Ayoub, O.: ChatGPT or human? Detect and explain. arXiv:2301.13852 (2023)

12. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)